DOCUMENTAÇÃO DA API

Rotas, parâmetros e exemplos de requisições

Projeto TCC — Backend desenvolvido com Node.js, Express, PostgreSQL e Drizzle ORM. Os exemplos abaixo mostram como consumir as rotas HTTP da aplicação.

1. Como utilizar a API

Substitua :id e outros parâmetros entre dois-pontos pelos valores reais. Nas requisições que enviam dados, utilize o cabeçalho Content-Type: application/json e envie o corpo no formato JSON.

URL base (exemplo): http://localhost:3000

Exemplo de cabeçalho: Content-Type: application/json

Importante: os caminhos e métodos foram organizados a partir das rotas revisadas no projeto. Alguns esquemas completos de entrada (campos obrigatórios e tipos) não estavam disponíveis no histórico consultado. Nesses casos, os corpos JSON são exemplos ilustrativos e devem ser conferidos com os respectivos arquivos schema/service antes da entrega final.

2. Resumo de todas as rotas

Módulo	Método e endpoint	Finalidade
Times	GET /times	Listar times
Times	GET /times/:id	Buscar time por ID
Times	POST /times	Cadastrar time
Times	PUT /times/:id	Atualizar time
Times	DELETE /times/:id	Excluir time
Posições	GET /posicoes	Listar posições
Posições	GET /posicoes/:id	Buscar posição por ID
Posições	POST /posicoes	Cadastrar posição
Posições	PUT /posicoes/:id	Atualizar posição
Posições	DELETE /posicoes/:id	Excluir posição
Usuários	GET /usuarios	Listar usuários
Usuários	GET /usuarios/:id	Buscar usuário por ID
Usuários	POST /usuarios	Cadastrar usuário
Usuários	PUT /usuarios/:id	Atualizar usuário
Usuários	DELETE /usuarios/:id	Excluir usuário
Giros	GET /giros	Listar giros
Giros	GET /giros/:id	Buscar giro por ID
Giros	GET /giros/escalacao/:escalacaoId	Listar giros de uma escalação
Giros	POST /giros	Registrar giro
Formações	GET /formacoes	Listar formações
Formações	GET /formacoes/:id	Buscar formação por ID
Formações	POST /formacoes	Cadastrar formação
Formações	PUT /formacoes/:id	Atualizar formação
Formações	DELETE /formacoes/:id	Excluir formação
Escalações	GET /escalacoes	Listar escalações
Escalações	GET /escalacoes/usuario/:usuarioId	Listar escalações de um usuário
Escalações	GET /escalacoes/:id	Buscar escalação por ID
Escalações	POST /escalacoes	Criar escalação
Escalações	PUT /escalacoes/:id	Atualizar escalação
Escalações	DELETE /escalacoes/:id	Excluir escalação
Escalações	PATCH /escalacoes/:id/finalizar	Finalizar escalação
EscalaçãoAtleta	GET /escalacao-atletas	Listar vínculos entre atletas e escalações
EscalaçãoAtleta	GET /escalacao-atletas/escalacao/:escalacaoId	Listar atletas de uma escalação
EscalaçãoAtleta	GET /escalacao-atletas/:id	Buscar vínculo por ID
EscalaçãoAtleta	POST /escalacao-atletas	Adicionar atleta à escalação

Campeonatos	GET /campeonatos	Listar campeonatos
Campeonatos	GET /campeonatos/:id	Buscar campeonato por ID
Campeonatos	GET /campeonatos/:campeonatoId/participantes	Listar participantes
Campeonatos	POST /campeonatos	Criar campeonato
Campeonatos	POST /campeonatos/participantes	Inscrever participante
Campeonatos	PUT /campeonatos/:id	Atualizar campeonato
Campeonatos	DELETE /campeonatos/participantes/:id	Remover participante
Campeonatos	DELETE /campeonatos/:id	Excluir campeonato
Autenticação	POST /auth/register	Registrar usuário
Autenticação	POST /auth/login	Realizar login
Atletas	GET /atletas	Listar atletas
Atletas	GET /atletas/time/:timeId	Listar atletas de um time
Atletas	GET /atletas/:id	Buscar atleta por ID
Atletas	POST /atletas	Cadastrar atleta
Atletas	PUT /atletas/:id	Atualizar atleta
Atletas	DELETE /atletas/:id	Excluir atleta

3. Detalhamento das rotas

3.1 Times — /times

GET /times

Finalidade: Retorna todos os times.

Parâmetros: Nenhum.

Corpo (JSON): Sem corpo.

Resposta esperada: 200 OK — lista de times em JSON.

GET /times/:id

Finalidade: Retorna um time específico.

Parâmetros: id: identificador do time na URL.

Corpo (JSON): Sem corpo.

Resposta esperada: 200 OK — objeto do time.

POST /times

Finalidade: Cadastra um novo time.

Parâmetros: Nenhum.

Corpo (JSON): {
  "nome": "Palmeiras"
}

Resposta esperada: 201 Created — mensagem "Time cadastrado com sucesso!" e dados do time.

PUT /times/:id

Finalidade: Atualiza os dados de um time.

Parâmetros: id: identificador do time na URL.

Corpo (JSON): {
  "nome": "Sociedade Esportiva Palmeiras"
}

Resposta esperada: 200 OK — mensagem "Time atualizado com sucesso!" e dados atualizados.

DELETE /times/:id

Finalidade: Exclui um time.

Parâmetros: id: identificador do time na URL.

Corpo (JSON): Sem corpo.

Resposta esperada: 200 OK — mensagem "Time excluído com sucesso!"

3.2 Posições — /posicoes

GET /posicoes

Finalidade: Lista as posições cadastradas.

Parâmetros: Nenhum.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

GET /posicoes/:id

Finalidade: Busca uma posição.

Parâmetros: id: identificador da posição.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

POST /posicoes

Finalidade: Cadastra uma posição.

Parâmetros: Nenhum.

Corpo (JSON): {
  "nome": "Atacante"
}

Resposta esperada: 201 Created — mensagem "Posição cadastrada com sucesso!" e dados da posição.

PUT /posicoes/:id

Finalidade: Atualiza uma posição.

Parâmetros: id: identificador da posição.

Corpo (JSON): {
  "nome": "Meio-campista"
}

Resposta esperada: 200 OK — mensagem "Posição atualizada com sucesso!" e dados atualizados.

DELETE /posicoes/:id

Finalidade: Exclui uma posição.

Parâmetros: id: identificador da posição.

Corpo (JSON): Sem corpo.

Resposta esperada: 200 OK — mensagem "Posição excluída com sucesso!"

3.3 Usuários — /usuarios

GET /usuarios

Finalidade: Lista os usuários cadastrados.

Parâmetros: Nenhum.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

GET /usuarios/:id

Finalidade: Busca um usuário.

Parâmetros: id: identificador do usuário.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

POST /usuarios

Finalidade: Cadastra um usuário.

Parâmetros: Nenhum.

Corpo (JSON): {
  "nome": "João",
  "email": "joao@email.com",
  "senha": "senha_de_exemplo"
}

Resposta esperada: 201 Created — mensagem "Usuário cadastrado com sucesso!" e dados retornados pelo controller.

PUT /usuarios/:id

Finalidade: Atualiza um usuário.

Parâmetros: id: identificador do usuário.

Corpo (JSON): {
  "nome": "João Silva",
  "email": "joao.silva@email.com"
}

Resposta esperada: 200 OK — mensagem "Usuário atualizado com sucesso!" e dados atualizados.

DELETE /usuarios/:id

Finalidade: Exclui um usuário.

Parâmetros: id: identificador do usuário.

Corpo (JSON): Sem corpo.

Resposta esperada: 200 OK — mensagem "Usuário excluído com sucesso!"

3.4 Giros — /giros

GET /giros

Finalidade: Lista os giros registrados.

Parâmetros: Nenhum.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

GET /giros/:id

Finalidade: Busca um giro pelo identificador.

Parâmetros: id: identificador do giro.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

GET /giros/escalacao/:escalacaoId

Finalidade: Lista os giros vinculados a uma escalação.

Parâmetros: escalacaoId: identificador da escalação.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

POST /giros

Finalidade: Registra um giro.

Parâmetros: Nenhum.

Corpo (JSON): {
  "escalacaoId": "UUID_DA_ESCALACAO"
}

Resposta esperada: 201 Created — mensagem "Giro registrado com sucesso!" e dados do giro.

3.5 Formações — /formacoes

GET /formacoes

Finalidade: Lista as formações cadastradas.

Parâmetros: Nenhum.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

GET /formacoes/:id

Finalidade: Busca uma formação.

Parâmetros: id: identificador da formação.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

POST /formacoes

Finalidade: Cadastra uma formação e suas posições relacionadas.

Parâmetros: Nenhum.

Corpo (JSON): {
"nome": "4-3-3",
"posicoes": [
{ "posicaoId": "UUID_DA_POSICAO_1" },
{ "posicaoId": "UUID_DA_POSICAO_2" },
{ "posicaoId": "UUID_DA_POSICAO_3" }
]
}

Resposta esperada: 201 Created — mensagem "Formação cadastrada com sucesso!" e dados da formação.

PUT /formacoes/:id

Finalidade: Atualiza uma formação.

Parâmetros: id: identificador da formação.

Corpo (JSON): {
"nome": "4-4-2",
"posicoes": [
{ "posicaoId": "UUID_DA_POSICAO_1" }
]
}

Resposta esperada: 200 OK — mensagem "Formação atualizada com sucesso!" e dados atualizados.

DELETE /formacoes/:id

Finalidade: Exclui uma formação.

Parâmetros: id: identificador da formação.

Corpo (JSON): Sem corpo.

Resposta esperada: 200 OK — mensagem "Formação excluída com sucesso!"

3.6 Escalações — /escalacoes

GET /escalacoes

Finalidade: Lista as escalações.

Parâmetros: Nenhum.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

GET /escalacoes/usuario/:usuarioId

Finalidade: Lista as escalações de um usuário.

Parâmetros: usuarioId: identificador do usuário.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

GET /escalacoes/:id

Finalidade: Busca uma escalação.

Parâmetros: id: identificador da escalação.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

POST /escalacoes

Finalidade: Cria uma escalação.

Parâmetros: Nenhum.

Corpo (JSON): {
  "usuarioId": "UUID_DO_USUARIO",
  "nome": "Meu time titular"
}

Resposta esperada: 201 Created — mensagem "Escalação criada com sucesso!" e dados da escalação.

PUT /escalacoes/:id

Finalidade: Atualiza uma escalação.

Parâmetros: id: identificador da escalação.

Corpo (JSON): {
  "nome": "Escalação principal"
}

Resposta esperada: 200 OK — mensagem "Escalação atualizada com sucesso!" e dados atualizados.

DELETE /escalacoes/:id

Finalidade: Exclui uma escalação.

Parâmetros: id: identificador da escalação.

Corpo (JSON): Sem corpo.

Resposta esperada: 200 OK — mensagem "Escalação excluída com sucesso!"

PATCH /escalacoes/:id/finalizar

Finalidade: Altera o estado da escalação para finalizada.

Parâmetros: id: identificador da escalação.

Corpo (JSON): Sem corpo.

Resposta esperada: 200 OK — mensagem "Escalação finalizada com sucesso!"

3.7 EscalaçãoAtleta — /escalacao-atletas

GET /escalacao-atletas

Finalidade: Lista os vínculos de atletas com escalações.

Parâmetros: Nenhum.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

GET /escalacao-atletas/escalacao/:escalacaoId

Finalidade: Lista os atletas associados a uma escalação.

Parâmetros: escalacaoId: identificador da escalação.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

GET /escalacao-atletas/:id

Finalidade: Busca um vínculo pelo identificador.

Parâmetros: id: identificador do vínculo.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

POST /escalacao-atletas

Finalidade: Adiciona um atleta a uma escalação.

Parâmetros: Nenhum.

Corpo (JSON): {
  "escalacaoId": "UUID_DA_ESCALACAO",
  "atletaId": "UUID_DO_ATLETA"
}

Resposta esperada: 201 Created — mensagem "Atleta adicionado à escalação com sucesso!" e dados do vínculo.

3.8 Campeonatos — /campeonatos

GET /campeonatos

Finalidade: Lista os campeonatos.

Parâmetros: Nenhum.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

GET /campeonatos/:id

Finalidade: Busca um campeonato.

Parâmetros: id: identificador do campeonato.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

GET /campeonatos/:campeonatoId/participantes

Finalidade: Lista os participantes de um campeonato.

Parâmetros: campeonatoId: identificador do campeonato.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

POST /campeonatos

Finalidade: Cria um campeonato.

Parâmetros: Nenhum.

Corpo (JSON): {
  "nome": "Campeonato TCC"
}

Resposta esperada: 201 Created — mensagem "Campeonato criado com sucesso!" e dados do campeonato.

POST /campeonatos/participantes

Finalidade: Inscreve uma escalação/participante em um campeonato.

Parâmetros: Nenhum.

Corpo (JSON): {
  "campeonatoId": "UUID_DO_CAMPEONATO",
  "escalacaoId": "UUID_DA_ESCALACAO"
}

Resposta esperada: 201 Created — mensagem "Participante inscrito com sucesso!" e dados da inscrição.

PUT /campeonatos/:id

Finalidade: Atualiza um campeonato.

Parâmetros: id: identificador do campeonato.

Corpo (JSON): {
  "nome": "Campeonato atualizado"
}

Resposta esperada: 200 OK — mensagem "Campeonato atualizado com sucesso!" e dados atualizados.

DELETE /campeonatos/participantes/:id

Finalidade: Remove uma inscrição/participante.

Parâmetros: id: identificador do registro de participação.

Corpo (JSON): Sem corpo.

Resposta esperada: 200 OK — mensagem "Participante removido com sucesso!"

DELETE /campeonatos/:id

Finalidade: Exclui um campeonato.

Parâmetros: id: identificador do campeonato.

Corpo (JSON): Sem corpo.

Resposta esperada: 200 OK — mensagem "Campeonato excluído com sucesso!"

3.9 Autenticação — /auth

POST /auth/register

Finalidade: Registra um usuário para autenticação.

Parâmetros: Nenhum.

Corpo (JSON): {
"nome": "João",
"email": "joao@email.com",
"senha": "senha_de_exemplo"
}

Resposta esperada: 201 Created — objeto retornado pelo serviço de registro.

POST /auth/login

Finalidade: Autentica um usuário.

Parâmetros: Nenhum.

Corpo (JSON): {
"email": "joao@email.com",
"senha": "senha_de_exemplo"
}

Resposta esperada: 200 OK — resultado do login, normalmente contendo os dados definidos pelo serviço (por exemplo, token e usuário, se implementados).

Observação: a rota /auth/me não foi incluída porque a versão de auth.routes.js revisada no histórico expunha apenas /register e /login. O método me existia no controller, mas não estava registrado no arquivo de rotas.

3.10 Atletas — /atletas

GET /atletas

Finalidade: Lista os atletas.

Parâmetros: Nenhum.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

GET /atletas/time/:timeId

Finalidade: Lista os atletas vinculados a um time.

Parâmetros: timeId: identificador do time.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

GET /atletas/:id

Finalidade: Busca um atleta.

Parâmetros: id: identificador do atleta.

Corpo (JSON): Sem corpo.

Resposta esperada: Resposta conforme o controller.

POST /atletas

Finalidade: Cadastra um atleta.

Parâmetros: Nenhum.

Corpo (JSON): {
  "nome": "Atleta Exemplo",
  "numero": 10,
  "timeId": "UUID_DO_TIME"
}

Resposta esperada: 201 Created — mensagem "Atleta cadastrado com sucesso!" e dados do atleta.

PUT /atletas/:id

Finalidade: Atualiza um atleta.

Parâmetros: id: identificador do atleta.

Corpo (JSON): {
  "nome": "Atleta Atualizado",
  "numero": 11
}

Resposta esperada: 200 OK — mensagem "Atleta atualizado com sucesso!" e dados atualizados.

DELETE /atletas/:id

Finalidade: Exclui um atleta.

Parâmetros: id: identificador do atleta.

Corpo (JSON): Sem corpo.

Resposta esperada: 200 OK — mensagem "Atleta excluído com sucesso!"

4. Exemplos de chamadas com cURL

Listar atletas

```
curl http://localhost:3000/atletas
```

Buscar atleta por ID

```
curl http://localhost:3000/atletas/UUID_DO_ATLETA
```

Cadastrar atleta

```
curl -X POST http://localhost:3000/atletas \
  -H "Content-Type: application/json" \
  -d '{"nome":"Atleta Exemplo","numero":10,"timeId":"UUID_DO_TIME"}'
```

Realizar login

```
curl -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"joao@email.com","senha":"senha_de_exemplo"}'
```

5. Códigos HTTP mais utilizados

Código	Significado	Quando aparece
200 OK	Requisição concluída	Consultas, atualizações, exclusões e ações concluídas.
201 Created	Recurso criado	Cadastros, criação de registros e inscrições.
400 Bad Request	Dados inválidos	Quando a aplicação rejeita dados enviados incorretamente, se houver validação.
401 Unauthorized	Não autenticado	Quando uma rota protegida exige autenticação.
404 Not Found	Recurso não encontrado	Quando o service/controller retorna erro de recurso inexistente.
500 Internal Server Error	Erro interno	Falha inesperada no servidor.

6. Observações importantes

- Os exemplos de UUID são marcadores: substitua-os por identificadores existentes no banco.
- Os corpos de requisição ilustram o formato e não garantem que todos os campos correspondam exatamente ao schema atual.
- A rota de autenticação /me não consta no arquivo de rotas revisado; só deve ser documentada depois de ser efetivamente registrada.
- A documentação descreve os endpoints revisados; middlewares globais, autenticação obrigatória e regras adicionais devem ser conferidos no arquivo principal da aplicação.
- Em rotas GET, parâmetros como id, timeId, usuarioId e escalacaoId são enviados na própria URL, não no corpo JSON.

Documento de referência — conferir os schemas e controllers antes da publicação.